

Ant-Game 通过例子学习

本节通过仓库自带的示例 AI，说明如何实现一个在 Ant-Game 环境下可运行的 AI，包括接口约定、动作构造、运行与评测。

建议先阅读「实验要求」与「规范」文档，再结合本节的例子动手实现自己的 AI。

1. 示例一：最小示例 AI (ai_example.py)

本实验指导使用的例子为「最小示例」基线：从候选动作中选出得分最高的非空操作（hold 除外），若无则选 hold。

核心逻辑提取如下（实际测试时请以完整文件为准）：

```
from SDK.utils.actions import ActionBundle
from SDK.backend import BackendState

class ExampleAgent(BaseAgent):
    def choose_bundle(self, state: BackendState, player: int, bundles: list):
        bundles = bundles or self.list_bundles(state, player)
        if len(bundles) <= 1:
            return bundles[0]

        # 从候选中选出得分最高者，优先非空操作
        shortlist = bundles[1 : min(len(bundles), 8)]
        best = max(shortlist, key=lambda bundle: (bundle.score, -len(bundle.operations)))
        return best or bundles[0]

class AI(ExampleAgent):
    pass
```

要点：

- 返回值：一个 `ActionBundle`，其 `operations` 为当回合要执行的操作序列。
- 候选生成：`self.list_bundles(state, player)` 使用 `ActionCatalog` 自动生成建造、升级、降级、超级武器、基地升级等候选。
- 合法性：`ActionCatalog` 只生成合法候选；最终发送前会再次由 `can_apply_operation` 校验。

2. 示例二：贪心 AI (ai_greedy) 简述

ai_greedy 在每回合内根据敌方压力、塔位价值、金币等，对候选动作打分，选择得分最高的组合。结构上同样是「从 list_bundles 得到的候选中选择一个 ActionBundle」。

通过对比例子 AI 与贪心 AI，可以理解：

- 如何从 state 读取塔、蚂蚁、金币、冷却等；
- 如何利用 ActionCatalog 生成多种操作类型（建造、升级、降级、超级武器、基地升级等）；
- 如何保证返回值格式统一（ ActionBundle ，内含 operations ）。

3. 接口与状态

3.1 choose_bundle 接口

```
def choose_bundle(  
    self,  
    state: BackendState,  
    player: int,  
    bundles: list[ActionBundle] | None = None,  
) -> ActionBundle:
```

参数	含义
state	当前游戏状态，类型为 SDK.backend.state.BackendState
player	己方玩家编号，0 为先手、1 为后手
bundles	候选动作列表；若为 None，可通过 self.list_bundles(state, player) 获取

返回： ActionBundle ，其 operations 为当回合要执行的操作列表。

3.2 从 state 中读取常用信息

信息	用法
己方塔	state.towers_of(player)
己方蚂蚁	state.ants_of(player)
敌方蚂蚁	state.ants_of(1 - player)

信息	用法
金币	<code>state.coins[player]</code>
当前回合	<code>state.round_index</code>
基地血量	<code>state.bases[player].hp</code> 、 <code>state.bases[0].hp</code> 等
建造塔成本	<code>state.build_tower_cost(state.tower_count(player))</code>
操作合法性	<code>state.can_apply_operation(player, op)</code>

更多字段与方法见「规范」中的 BackendState 一节。

4. 操作构造

操作通过 `SDK.backend.model.Operation` 构造，协议格式统一为：第一个元素为操作类型，其后为参数列表。常用示例：

类型	含义	示例 (Operation / 协议)
11	建造防御塔	<code>Operation(OperationType.BUILD_TOWER, x, y) → 11 x y</code>
12	升级防御塔	<code>Operation(OperationType.UPGRADE_TOWER, tower_id, type_id) → 12 towerId typeId</code>
13	降级/拆除塔	<code>Operation(OperationType.DOWNGRADE_TOWER, tower_id) → 13 towerId</code>
21	闪电风暴	<code>Operation(OperationType.USE_LIGHTNING_STORM, x, y) → 21 x y</code>
22	EMP 轰炸	<code>Operation(OperationType.USE_EMP_BLASTER, x, y) → 22 x y</code>
23	引力护盾	<code>Operation(OperationType.USE_DEFLECTOR, x, y) → 23 x y</code>
24	紧急回避	<code>Operation(OperationType.USE_EMERGENCY_EVASION, x, y) → 24 x y</code>
31	升级生产流水线	<code>Operation(OperationType.UPGRADE_GENERATION_SPEED) → 31</code>
32	升级高级护甲	<code>Operation(OperationType.UPGRADE_GENERATED_ANT) → 32</code>

所有操作由 `BackendState.can_apply_operation()` 校验；非法操作会返回 False 且不修改状态。参数范围与规则见「规范」文档。

5. 运行与评测

5.1 本地对战与回放

```
python tools/run_local_match.py --ai0 greedy --ai1 random --seed 7
```

回放默认写入 `--keep-dir` 指定目录下的 `replay.json`。若你的 AI 名为 `my_ai`：

```
python tools/run_local_match.py --ai0 my_ai --ai1 greedy --seed 7 --keep-
```

5.2 打包 AI

评测前需打包 AI，将指定 AI 与 SDK 等依赖放入输出目录：

```
bash AI/package_ai.sh <ai_name> <output_dir>
```

例如打包 example：

```
bash AI/zip_example.sh
```

打包后目录包含 `main.py`（协议入口）、`ai.py`（当前 AI）、`SDK/` 等。

5.3 训练脚本

```
bash SDK/train_example.sh --seed 1 --max-actions 16
```

```
bash SDK/train_mcts.sh --episodes 2 --iterations 24 --max-depth 3 --seed
```

支持自定义回合数、深度、种子等参数。详见仓库 README 与「实验要求」文档。

5.4 单元测试

```
pytest -q
```

提交前请确保通过，其中包含 AI 接口与短局对战相关测试。

6. 实现建议

1. **先保证接口正确**：继承 `BaseAgent`，实现 `choose_bundle`；返回值必须是 `ActionBundle`，不操作时可返回 `bundles[0]`（`hold`）。
2. **从简单策略开始**：例如只选 `hold`，或仅从建造塔候选中选一个，对照 `ai_example` 检查操作与参数。
3. **善用已有逻辑**：如 `ActionCatalog.build()`、`state.can_apply_operation()`、`state.towers_of()` 等，避免重复造轮子。
4. **合法性**：建造、升级、降级、超级武器、基地升级均有规则与资源限制，非法操作会被拒绝；调试时可通过 `tools/run_local_match.py` 查看回放与 `stderr` 日志。